

Where Pathfinder Hurts

Shortcomings, a cross-repository failure catalogue, and directions for reducing friction

Pathfinder operator notes

29 September 2026

Abstract

This note consolidates two things: the operator’s list of what does not work satisfactorily in Pathfinder, and a read-only post mortem of run logs from four repositories that ran Pathfinder or its ancestors. The post mortem covers about 2,100 receipts in pathfinder, 115 in statarb, 481 dispatches in pathfinder-julien-2 and about 1,500 agent calls in agQSL. Its central finding is that failures are either not classified at all or classified by grepping noisy stderr, so the supervisor and the operator learn about them late or never. The note ends with directions for improving the code and a list of questions to settle. It is meant to open that discussion, not to close it.

Contents

1	Scope and sources	2
2	Root causes in brief	2
3	Shortcomings reported by the operator	3
3.1	Failure monitoring and reporting (R1, R2)	3
3.2	Apex agent (R5)	3
3.3	Visibility into campaign progress (R2)	3
3.4	Context and memory (R3)	3
3.5	Prefix caching (R3)	3
3.6	Code quality and architecture (R4, R7)	3
3.7	The operator loop is not normalised (R2)	3
3.8	Value of repetition	4
4	Failure catalogue	4
4.1	How classification fails today (R1)	4
4.2	Provider side	4
4.3	Launch and runtime environment	5
4.4	Context size	5
4.5	Output contract and parsing	6
4.6	Budget and loops	6
4.7	Error reporting fidelity	7
4.8	Supervision, state and alerting	7
4.9	Verification quality	8
5	Directions for improvement	8
6	Harvest from statarb and agQSL	9
6.1	Engine work to take or adapt	10
6.2	Dropped	10
6.3	Migration	11

1 Scope and sources

Scope is detection and visibility. Recovery is deliberately left for later: a failure that is not classified cannot be recovered from reliably.

PF: pathfinder

About 2,100 receipts, from the pilot of 11 September through the recursive extension of 28 September.

SA: statarb **arxiv_drip**

23 runs, 115 receipts, 23 to 29 September.

J2: pathfinder-julien-2

481 dispatches. Pilot on 25 September; the top-ten batch of 28 and 29 September made 208 charged attempts and ended with all 10 pairs blocked.

AQ: agQSL

Pathfinder3 phases A to E, the Julien edition and phase E prime: about 1,500 agent calls, 1,763 campaign events and 284 phase B failure rows, 20 August to 14 September.

In the tables, *yes* means classified and surfaced automatically, *partial* means recorded but generic, misclassified or found only after the fact, and *no* means invisible without manual digging.

2 Root causes in brief

The friction reported by the operator and the failures in the logs trace back to a short list of structural causes. Each later section refers back to these.

- R1 No error taxonomy.** Provider and runtime failures collapse into a generic error, or are classified by regex over stderr tails that are drowned in noise. Supervisors recognise almost nothing.
- R2 No shared event format or campaign state.** Each component logs and reports health in its own way. Neither the operator nor the apex agent has a single view of where a campaign stands.
- R3 Context is inlined, not referenced.** Ledgers and evidence are pasted into prompts, which overflow input limits, inflate cost and defeat prefix caching.
- R4 Brittle output contracts.** Agent output is checked by large, special-cased parsers and provenance validators that produce both real and false blocks, each fixed by a new patch.
- R5 A supervisor that notices but does not act.** The apex agent knows what to do but does not do it, alerts are lost or repeated, and the supervisor shares a failure domain with the workers.
- R6 Unreliable accounting.** Budgets cannot trip under subscription billing, receipts are double counted, and operational exhaustion is recorded as a scientific outcome.
- R7 Lessons do not propagate.** The code is not modular enough to share fixes between lineages. AQ already had quota detection, a health policy that retries once and probes with one seat, and a size preflight; SA had its own size preflight and recovery diagnoser; canonical PF has none of these, while it does have a 60 s no-start grace that AQ also built separately. In the same week, main and julien-2 each moved evidence out of prompts, independently and in incompatible ways.

3 Shortcomings reported by the operator

3.1 Failure monitoring and reporting (R1, R2)

Failures are not monitored systematically. New failure modes keep appearing and are discovered by hand, after the fact. The operator named provider censorship (biology questions in the second Julien campaign), token allowance exhaustion, server congestion, agents not talking well to parsers, internal loops such as the research phase running over budget, and process launch troubles that do not bubble up. All of these are confirmed by the post mortem in Section 4, along with many more.

3.2 Apex agent (R5)

The agent trusted with overnight runs does not recover from failures, even when notified and launched with an explicit “be autonomous” prompt. Asked afterwards, it always knows what it should have done. The gap is between diagnosis and action. Candidate causes: model choice, prompt, harness (notifications that do not start a new turn, or a turn that ends waiting for a human), and missing permissions. When it does act, it may act blindly: on 29 September (SA) it restarted a stopped batch without diagnosing the repeated error, and the batch stopped again on the same blocker. The logs show it idle for about 39 hours (PF, 25 to 27 September), about 5 hours (J2, night of 28 September, until the operator typed “fix and resume all”), and 10 hours (AQ, 9 to 10 September).

3.3 Visibility into campaign progress (R2)

There is no normalised way to see which phase or thread is running, what has completed, what is stuck or failed, and how much budget has been spent against plan. Two consumers need that view. The operator currently digs through logs and run directories, differently for each version. The apex agent reconstructs the state by hand each time, which is costly and may be part of why it does not act. The same machine-readable state should serve both.

3.4 Context and memory (R3)

Agents are handed everything inline. On 28 September Vera, the verifier in the EVA loop, received a 9.1 million character request (1.2 million input tokens). There is no way for information to live outside the context and be pulled in on demand, through reading tools or a structured memory the agent can query.

3.5 Prefix caching (R3)

Prompts are not arranged so that stable material forms a common prefix that provider caching can reuse. Repeated calls in loops such as EVA pay full input price.

3.6 Code quality and architecture (R4, R7)

The code has grown by accretion. Parsers are bloated because each agent/parser mismatch adds a patch rather than tightening the output contract. Monitoring is heterogeneous across components. Pathfinder is not modular enough to swap, test or reuse pieces, for example the repetition scheme or the apex agent.

3.7 The operator loop is not normalised (R2)

Working with Pathfinder follows a recurring loop, comparable to a REPL: report on what a run did, recap the state, decide and apply fixes, choose next steps, run again. Each turn of this loop is currently improvised: its inputs are gathered by hand, its output has no fixed shape, and nothing carries one iteration’s decisions into the next. The operator wants the loop captured as a skill with a normalised format, so that every iteration looks the same whether the operator or the apex agent runs it.

3.8 Value of repetition

The new version runs EVA three times independently and then synthesises the three threads. Whether this is a good use of budget has not been measured. Open points: whether the threads actually diverge; whether diversity should be designed in (different models, prompts or angles) rather than left to sampling; whether repetition should be adaptive, adding threads only when confidence is low or threads disagree; whether threads should exchange findings midway; and whether the synthesis keeps minority findings.

4 Failure catalogue

4.1 How classification fails today (R1)

- PF records only completed, error, timeout, no session or launch failed plus free text (pathfinder/transport.py:327-335); health.py counts error rows.
- J2 reports nearly everything as Runtime exit N; see retained stderr and classifies only the biology filter, and only in top-ten mode.
- SA's supervisor recognises two signatures and labels the rest unrecognized_operational_failure: 10 of 12 blocked outcomes.
- AQ went furthest but greps the last few thousand characters of stderr, which hold about 2,650 lines of model-list refresh noise plus thread slugs and task text. A slug containing quantum-memory-capacity tripped the capacity rule, and 10 of 16 capacity calls show no capacity wording. Elsewhere in AQ only the exception class or return code survives.

4.2 Provider side

Mode	Signature	Seen	Detected	Notes
Safety refusal	safeguards flagged this message; flagged for possible biological risk; provider_response_rejected	PF 09-11 (3); AQ 09-06 (3 pairs); J2 09-28, Q016 x P011 (2 of 3 branches)	partial	Nondeterministic: identical branches or retries pass. Fires on prompt content before tool use. J2 stop file written by a human. In AQ dependent pairs never started and nobody noticed for 5 days.
Quota exhausted	You've hit your usage limit ... try again at Oct 4th; You've hit your session limit	AQ 09-06 to 09-10 (6); PF 09-19 (41 in 24 s); J2 09-29 (11 calls, 6 pairs in 76 s)	AQ yes; PF, J2 no	No global stop in PF or J2: every queued pair launches and fails. Reset time is in the message but not parsed.
Quota disguised as rate limit	HTTP 429 after a server-directed pause	AQ 08-26 (whole ELM phase A)	partial	Real cause found by hand; ELM spend counter lags, so cost guards read stale spend.
Rate limit	rate_limited	AQ Aug, 09-06 (16)	yes	None in PF, SA, J2.
Capacity, 503, disconnect	Selected model is at capacity; 503 Service Unavailable; stream disconnected before completion	AQ 09-07 to 09-12 (819 lines); PF (6); SA 09-25 (5)	partial	Retried inside the CLI with no host counter. In AQ a 503 tail after a written response made finished calls look failed.
Auth failure	401 Unauthorized: Incorrect API key provided; Missing environment variable: ELM_API_KEY	PF 09-19 (40), 09-25; SA 09-25; AQ 09-09	partial	Retried 5 times as transient. No preflight. PF supervisor shared the key and died too.
Key file misconfigured	ApiKeyError	AQ 08-31 (100 in one lane)	partial	Checked per call, not before the batch.
Network and DNS	gaierror	AQ 08-30/31 (172)	partial	Stored as class name only.

Mode	Signature	Seen	Detected	Notes
Silent model switch	Observed model differs from pinned route; <code>ModelMismatchError</code>	J2 09-25; AQ Aug (6), 09-10	partial	Failed model still billed.
External services	arXiv HTTP 406, HTTP 429; <code>latexpd</code> crash; missing <code>.sty</code>	PF (about 20 pairs); SA intake (2)	yes	No retry or backoff in SA intake.
Market data API timeouts	<code>api.hyperliquid.xyz</code> read timeout (10 s, 45 s)	SA 09-30 (22 blocked steps over 4 h)	partial	API slow, not down (200 in 1.5 to 7.6 s). No retry until fixed on 09-30. Blocks paper-trading steps of live experiments.
Neighbouring services unmonitored	database disk image is malformed every minute; quotes 18,000 s stale	SA Polymarket services, 09-30	no	Noticed only in passing by the recap agent.

4.3 Launch and runtime environment

Mode	Signature	Seen	Detected	Notes
Fast repeated launch failure	<code>transport_process_error</code> , return code 1, 0 bytes, 0.02 s	AQ 09-05/06 (164 in one run)	no	No stderr kept; nothing stopped the retries.
CLI flag drift	<code>unexpected argument '--search' found</code>	PF 09-23 (22); SA 09-24 (2)	partial	SA shows only <code>TransportFailed</code> . PF health probe used other arguments and logged health restored 4 times.
CLI and toolchain drift	Runtime version changed: <code>codex</code> ; Codex 0.154.0 hanging in the loader	J2 (2); AQ 09-10 (3 episodes)	J2 yes; AQ no	CLI auto-updated mid-campaign. AQ pinned binary lived in <code>/private/tmp</code> and a reboot wiped it.
Silent hang	no events, timeout, 0 tokens at 900 s	AQ 09-10 (60)	no	About 1 h to diagnose; AQ then added a 60 s no-start timeout. The only true hangs found.
Child segfault	<code>exit -11 before session</code>	PF 09-23 (65 in 60 min)	partial	Probe loop had no deadline. Workaround <code>no_proxy=*</code> .
No session in grace	<code>outcome no session</code>	PF 09-19 (241)	partial	Cause not diagnosed.
Sub-agent spawn swallowed	<code>collab spawn failed: no thread with id</code>	SA 09-23/24 (3 of 3 legacy runs)	no	Multi-agent research silently ran single-agent.
Sandbox and tool friction	<code>Operation not permitted</code> ; No module named <code>'pathfinder'</code> ; File name too long	J2 (about 2,000 lines); SA (3.3% of tool commands); PF, AQ (about 20)	no	Agents burn turns on workarounds; <code>exit 0</code> hides it.
Approval reviewer blocks launch	Automatic approval review rejected launch	J2 09-28	yes	Unattended launch waits on a human.
Lock contention	<code>waiting_for_pipeline_lock</code>	SA 09-28	yes	No timeout for a stale lock.
Background launch dies silently	PID recorded, no process, empty worker log, queue still prepared	SA 09-29 19:3x	no	The launch PID was taken as evidence that reviews were running. Relunched as a detached session with redirected streams.
Nested sandbox blocks the apex agent	pipeline's strategy sandbox raises <code>PermissionError</code> inside the agent sandbox; request to disable the sandbox denied by the permission classifier	SA 09-30	yes	The agent could not resume the batch it supervises and handed a launch command back to the operator.
Misread process state	<code>launchctl nested state = active</code>	AQ 08-21	no	

4.4 Context size

Mode	Signature	Seen	Detected	Notes
Hard input limit	Input exceeds the maximum length of 1048576 characters	SA 09-28 (1, plus 4 just under); J2 (3, up to 9.1 M chars); AQ 09-07	partial	SA and AQ phase B have a preflight; main Pathfinder does not.
Structured data pasted whole	review prompt 1,272,855 chars; one preflight event of 90 cases (605,835 chars of JSON)	SA 09-29 19:54	yes (preflight)	The size gate worked, but the batch stopped. Pooling only deduplicates long strings, not large objects. Fixed by abridging long lists with a count and hash.
Unbounded prompt growth	Vera request 9.1 M chars, 1.2 M tokens; peer prompts up to 91 M chars	J2 09-29; AQ (4 to 5 M tokens)	no	Ledger and evidence inlined. J2 repair 02 routes oversize through read-only packets instead of capping. Cost grows through prompt size, not call count.

4.5 Output contract and parsing

Mode	Signature	Seen	Detected	Notes
Unparseable reply	no JSON object in reply; consolidate: no note; JSONDecodeError; InstrumentError	AQ 08-31 (134); PF (15); J2 (6)	yes	Not distinguished from a scientific outcome. Each slip fixed by a manual patch.
Provenance and binding checks	artefact digest mismatch; complete artifact contradicts inline result; Missing local evidence referenced by; Aliased evidence directory	AQ 09-08 to 09-12 (155 deferred, 106 failed); J2 (blocked about 8 of 10 pairs)	yes, often false	Largest single class. J2 reads equations, ratios and DOIs as paths, fixed three times. AQ: one pair needed 6 passes, each fixing one binding and breaking the next.
Schema-valid, contract-invalid	READY needs code and feeds within the configured allowance	SA 09-29 (9)	partial	Agent proposes feed-free studies; message wrongly suggests an allowance overflow.
Style gate, no repair	Banned style found	SA 09-29	partial	Discards a 6-minute call.
Style gate on factual text	counted framing rule fired on a feed inventory	SA 09-29 (Alpha046)	partial	The rule targets rhetoric but matched a factual list of eight feeds.
Hash basis mismatch	receipt hashes the saved file, reviewer hashes the source string	SA 09-30 (2609.28463v1)	no	One trailing newline paused a study the reviewer itself called coherent. Receipts now carry both hashes and state the rule.
Over-strict candidate check	exact floating-point equality on a normalisation	SA 09-29 (2609.25617v1)	partial	Needed a tolerance of 10^{-12} .
Agent-written patch broken	indentation error in the host patch	SA 09-29	partial	Caught before execution.
Non-UTF-8 output	UnicodeDecodeError('utf-8' from latexmk	SA 09-28	yes	Crashes the runner.

4.6 Budget and loops

Mode	Signature	Seen	Detected	Notes
Research allowance exhausted	allowance ran out before every peer declared ready; PAUSE-ON-ITERATE	PF 09-13 to 09-28 (about 15); AQ 09-13	partial	Recorded as a scientific state. PF guard checks only at admission; one run went 6.58 USD over.
Hard budget kill	CEILING REACHED: stopping coordinators	AQ (at USD 1,019)	yes	Left 8 orphaned calls and dead reservations.
Timeout too tight	timeout at 600 s or 300 s	PF 09-11 (2); AQ (1)	yes	Consolidation needed up to 1,344 s.

Mode	Signature	Seen	Detected	Notes
Budget unenforceable	<code>cost: None</code>	SA (115 of 115)	no	A 60 USD ceiling can never trip.
Metering and double counting	<code>recovery copies</code> <code>receipts.jsonl</code>	PF 09-25; SA 09-29	no	Naive aggregation double counts.
Refusals and cancellations charged	<code>quota refusals hit the attempt cap</code> ; <code>Runtime exit -15</code>	AQ; J2 09-28 (4)	partial	Counted against ceilings.

4.7 Error reporting fidelity

Mode	Signature	Seen	Detected	Notes
Message discarded	only exception class or return code	AQ (284 rows and runner events)	no	AQ recovery runs reached r9, suggesting blind retries.
Stored error is noise	last 1,500 chars of <code>stderr</code> ; <code>Reconnecting...</code> 2/5 stored as final	AQ (most calls); PF (3)	no	
Cancellation hides cause	<code>BLOCKED: Runtime exit -15</code>	J2 09-28	no	Names the SIGTERM, not the refusal.
Pointer to empty file	see <code>implementation-stder</code> <code>r.log</code> (0 bytes)	SA 09-25	no	Real error only in <code>events.jsonl</code> .
Corrupted JSONL	raw <code>Traceback</code> inside <code>runner-*.jsonl</code>	AQ (9 of 18 files)	no	Breaks consumers.
False success	<code>"finished": true, "error": null</code> with 14 incomplete pairs	PF 09-23	no	
False failure	503 tail after a written response	AQ	no	Needed a salvage flag.
Crash after completion	<code>FileNotFoundError</code> with no path	SA 09-25	partial	
Duplicate incidents	69 records for 7 incidents	PF	n/a	Inflates counts.
Stale error field	<code>READY</code> candidate keeps its preparation-stage error message	SA 09-29	no	Completion changes status without clearing the error.

4.8 Supervision, state and alerting

Mode	Signature	Seen	Detected	Notes
Apex does not act	<code>supervision_needs_operator</code> ; <code>finished_with_blocks</code> ; <code>stopped_repeated_operational_failure</code>	PF (39 h); J2 (5 h); SA (2 stops); AQ (10 h)	detected, no action	SA failure counter resets on restart.
Blind restart	agent's own recap: "I should have diagnosed the repeated error before restarting the batch"	SA 09-29 12:30 (second stop on the same <code>READY</code> /feeds blocker)	no	The supervising agent resumed a stopped batch without diagnosing why it stopped, so the same contract mismatch consumed two more investigations. Nine identical errors were treated as nine independent failures. Nothing requires a diagnosis before a restart.
Resume path not idempotent	resume loop raised on rows already stopped and refused to start from a blocked queue	SA 09-30	no	The previous recap had declared that no recurring execution blocker remained; the resume path had never been exercised. A resume must be dry-run before being declared ready.

Mode	Signature	Seen	Detected	Notes
Admission window expiry	4-hour admission window lapsed during a stop	SA 09-30	yes	Resuming needs a fresh operator approval; the apex agent cannot extend it.
Wrong state, alert fatigue	authentication blocked repeated	AQ 09-09/10 (110 alerts in 10 h)	detected, wrong	12 of 13 blockers were really missing output.
Alert delivery lost	RuntimeError: target unavailable	J2 09-29 (10 to 11)	no	Why the quota cascade went unnoticed.
Stale running status	may be stale after controller exit	J2 (3 branches); PF	partial	No liveness reconciliation.
Stale checkpoint or review	Checkpoint already exists with different content	AQ 09-11 to 09-13	partial	Pre-repair review reused.
Retry discards steering	retry replaced the resumption brief	AQ 09-12	no	Corrective brief lost.
Starvation and slow pipeline	one pair leased per thread; one export per 10 min	AQ 09-10, 09-13	partial	6 h idle threshold.
Retry storms	bursts of identical errors	PF, AQ, J2	no	Up to 196 failures in an hour, no rate alarm.
Shared failure domain	same 401 kills supervisor and workers	PF 09-25	no	Same credentials and transport.

4.9 Verification quality

Mode	Signature	Seen	Detected	Notes
Verifier never rejects	DRAFT, round: 1; verify calls 10 to 28 s, under 550 output tokens	SA (16 of 16 runs)	no	Reject and revise paths never exercised. Possibly legitimate; needs a test.

5 Directions for improvement

These are starting points for discussion, ordered by how much friction each removes per unit of work. Each names the root causes it addresses.

D1 One event schema and a derived campaign state (R2). Every component appends typed events to one strict JSONL stream per campaign: call started, completed, failed with class, stage transitions, budget spend. A small reducer turns the stream into a campaign state file. The operator gets a single status command over it; the apex agent reads the same state through a tool. This also fixes stale statuses, duplicate incidents and corrupted logs.

D2 A shared error classifier (R1). Classify from the structured events the CLIs already emit (turn .failed, stream JSON), never from stderr tails. Classes: refusal, quota (with parsed reset time), rate or capacity, auth, input too large, launch or argv, no session, timeout, cancelled (with its cause), parse, contract, environment. Each class carries a scope (call, pair, campaign), so quota and auth become campaign-wide conditions. The catalogue's signatures become the test fixtures.

D3 Preflight before a batch (R1, R7). Credentials, pinned CLI version and flags, toolchain outside /private/tmp, prompt size against the limit. AQ already has most of this.

D4 Context by reference (R3). Prompts cite files and agents read them through tools; a size cap raises an event instead of silently packaging. Order prompts as stable prefix then variable tail to benefit from caching.

Status on 29 September: main commit 81f888e moves single-engine consolidation and verification to file tools, and julien-2 has a reviewed counterpart for the joint EVA stage (handover in plans/2026-09-29-1758-julien-2-joint-evidence-by-reference-handover.md): a hash-bound

index of evidence files, a read-only sandbox for the readers, and reference validation kept. On the retained Q002-P024 joint thread, review material falls from 1.0 M to 0.34 M characters, almost all of it the two papers. The two differ on read-only versus writable tools and on keeping validation, and account, editor and fidelity stages still inline the whole thread.

- D5 Contracts over parsers** (R4). One declared schema per stage, validated once, with a single repair turn on violation. Contract failures become an operational class, separate from scientific outcomes. Provenance checks validate declared references only, not paths extracted by regex.
- D6 A supervisor that acts** (R5). Wake on events rather than polling; read the campaign state; follow a playbook mapping error class to action; deduplicate alerts and confirm delivery; run on credentials and a transport independent of the workers.
- D7 Accounting in tokens** (R6). Meter tokens as the primary unit under subscription billing, deduplicate receipts by call identifier, and charge cancellations separately.
- D8 One upstream, thin deployments** (R7). Transport, classifier, event log and state as library modules that julien-2, statarb and future campaigns import rather than fork. This is where the tidy and simplify sweep belongs.
- D9 A skill for the operator loop** (R2, R5). Drafted on 29 September as repres (report and resume), shared by Claude Code and Codex from `~/codex/skills/repres`. Codify the report, recap, fixes, next steps cycle as a skill with a fixed output template. Report and recap are generated from the campaign state of D1 and the failure classes of D2, not assembled by hand; fixes and next steps are recorded as structured entries that the next iteration reads back. The same skill serves the operator interactively and the apex agent overnight, which also gives the apex agent an explicit procedure to follow when something fails.

A first template, extracted from a typical recap the operator received on 29 September (SA, 13:11 Paris), with gaps in that recap added:

1. *Status* with timestamp: running or stopped, since when, and why, in one sentence; whether any agent is running.
2. *Totals* as fixed counters from the campaign state, with the change since the previous iteration (the example gave totals but no delta).
3. *Blockers*, grouped by failure class with counts, so that nine instances of one cause read as one blocker.
4. *Corrections*: what the previous iteration got wrong.
5. *New results* since the previous iteration.
6. *Still active*: live items with their health measures.
7. *Fixes applied* in this iteration (missing from the example).
8. *Fixes to apply next*: repairs to code, contracts or artefacts that are diagnosed but not yet done, each tied to the blocker it removes.
9. *Next steps*: campaign actions (resume, relaunch, pause), ordered, each with a precondition to check before acting; a restart requires a diagnosed blocker and its fix applied.
10. *Decisions needed from the operator* (missing from the example).

- D10 Measure before redesigning EVA**. Measure divergence across the three EVA threads, and test Vera by planting flawed notes to see whether it rejects them.

A suggested order: D2 and D1 together first, since they are the detection foundation and fit the current scope; then D3 and D4, which are cheap and remove whole failure families; then D5 and D6; with D8 as the frame for the sweep.

6 Harvest from statarb and agQSL

Read-only exploration on 29 September of statarb's `arxiv_drip` and of agQSL's latest generation (phase E prime with the Julien transport and the pathfinder3 peer coordinator, last code commit 14 September;

older generations ignored), each compared with canonical pathfinder at 81f888e.

6.1 Engine work to take or adapt

Item	Mechanism	Source	Canonical today, and what to do
H1	Failure classes from structured events: quota, auth, rate, with deployment-registerable rules	AQ QuotaPaused; SA recovery diagnoser	Only generic outcomes. Classify from <code>turn.failed</code> payloads, never <code>stderr</code> . Quota stops admission with reason <code>quota</code> and does not BLOCK the thread; auth stops; 429 defers. (D2)
H2	Prompt size gate before admission, refusal receipt, lossless JSON compaction, pooling of large sources	SA <code>prompt_context.py</code> ; AQ input hygiene	Only a word count at the edit stage. Take into <code>transport.execute</code> . (D3, D4)
H3	Health policy: retry once, flag after two consecutive failures, auto-clear, single-seat probe	AQ runner	Any stage exception drains the run; restart is manual. Adapt. (D6)
H4	Per-call last-event time	AQ runner	<code>health.snapshot</code> has heartbeat and overdue calls but no event age. Stamp it in <code>active-calls/*.json</code> . (D1)
H5	Coordinator batch mode: deadline, maximum units, stop marker, opt-in tolerance of k consecutive failures, recovery first, deployment <code>next_unit()</code>	SA <code>overnight.py</code>	<code>coordinator.py</code> runs a fixed schedule and stops on the first failure. Adapt, opt-in.
H6	Recovery: one admission per attempt; reconcile actions to collect retained verify output, retry a stage refused before any turn, and mark terminal failure	SA <code>recovery.py</code> ; AQ <code>reconcile.py</code>	<code>reconcile.py</code> has four actions, <code>recovery.py</code> only repairs verdict JSON. Adapt.
H7	Cross-process seat pool keyed by account, reservations bound to PIDs, dead ones reaped	AQ <code>coordination.py</code>	Admission counts within one process. Needed before statarb and agQSL share one subscription.
H8	Schema-constrained single calls	SA (<code>codex exec --output-schema</code>)	<code>transport.call</code> has no schema support. Add; statarb's scan, implementation and review then use the engine.
H9	Post-edit consumer stage, which cannot overturn the verifier	SA implementation stage	No such hook. Adapt as an extension point.
H10	Monitor: Host and Origin checks, CSP, file allowlist and size cap; aggregate across campaigns; per-stage unit counts	SA watchboard	<code>monitor.state()</code> already has parameters, scores, blocks, stages and PDF links, and is the base for the operator page; its server serves any file under the root.
H11	Incremental intake: watermark advanced on success, dated negative cache	SA intake	<code>corpus.fetch</code> makes one request with no retry.
H12	Test that a repair survives a transport failure during consolidation	AQ	Behaviour already correct in <code>research.py</code> ; add the contract test.

Later candidates: an append-only event log with replay validation (pathfinder3, an older AQ generation; aligns with D1) and AQ's full-text acquisition as a corpus adapter.

6.2 Dropped

statarb's engine compatibility shim (its fix is already at HEAD), the deploy hold by monkeypatching, two one-off scripts, copy-then-driver recovery and its direct model calls once H8 lands. From agQSL: the external cost guard (harmful: it once killed the coordinator and orphaned 8 calls), the `stderr` regex

classifier, salvage-transport and the Julien watcher (superseded at HEAD), and the ELM route pin (canonical has the backend).

6.3 Migration

statarb. Stays in statarb as deployment code: intake policy, scanner and memory, mandate and briefs, the READY and feeds contract, review, deployment bundle, paper trading and its scheduling, data access, and the watchboard panels as `snapshot_extra`. Steps: its owner commits the pending `arxiv_drip/work`; land H2, H6, H10 and the classifier; add H8 and H9 with a deployment test on the stub backend; add H5; tag `engine-v1.2` and repin; replace `overnight.py` with a batch schedule; retire the watchboard server once the operator page exists. Risks: archived campaigns on frozen v1.1 engines; verdict drift from 81f888e while statarb sets inline evidence, to be measured before repinning; “paper” means paper trading in statarb and the manuscript in the engine; two lock regimes during the transition; hard-coded host paths.

agQSL. Becomes a campaign repository pinning a frozen engine release. Domain content stays: question and paper sets, papers, prompt overlays, styles, notes and published supplements. Steps: tag the current head `pre-pathfinder-roof`; land H1, H3, H4, H6 and H7; map the ITERATE/PAUSE/DRAFT decision words through a prompt overlay; import only the open pairs into canonical thread directories; pilot two pairs; add agQSL to `deployments.toml`. Archive, never delete: tracked production artefacts stay reachable through the tag; untracked build output and the full-text cache (3.7 GB) go to a compressed archive with a checksum manifest. Risks: ledger schema translation, paths cited by published supplements (leave redirect notes, do not move), the ELM key held outside the repository, and account oversubscription until H7 lands.

7 Refactor specification additions from statarb fixes

Fixes applied to statarb on 30 September (commit 4466744 there) that generalise to the canonical engine and belong in the refactor specification.

- S1 Context by reference, lossless and budgeted** (D4, H2). Pool repeated strings; write every document over 16k characters, the largest others beyond a 64k inline budget, and every JSON subtree over 16k characters to `context/<sha256>.txt` or `.json`; list them in an index with size and digest; resolve nested references. The reading stage gets a read-only shell. A test reconstructs the full value from prompt and files. Effect on a real handoff: 1.27 M characters to 83 k.
- S2 Agent sessions multiply their prompt.** A peer session re-reads its context on every turn: one peer call recorded 2.4 M input tokens, 94 % cached, for a prompt that inlined a 335 kB paper. Inline flags for papers and ledger must default to off, and receipts should record prompt size, turns and cached share so the amplification is visible.
- S3 A sanctioned launcher that verifies liveness.** Starting a batch goes through one command that detaches it in its own session, keeps one PID across shell handover (`exec`), waits, and fails loudly with the log tail if the process died. No hand-written `nohup` recipes. This is also the route to allow for the apex agent, whose own sandbox otherwise prevents it from launching pipelines that need their own sandbox.
- S4 Watch by PID and polling.** Name-based process checks break when a shell execs its last command; following a log with `tail -F` can deliver nothing. Supervisors poll line counts and check the PID.
- S5 State transitions clear stale errors.** Every status change writes the error field explicitly, so a completed unit never shows an old failure.
- S6 Derived stores heal themselves.** A cache that fails to open is moved aside with a timestamp and rebuilt; the failure is logged once, not every minute for two weeks.

- S7 Controlled clocks in tests.** Anything gated on wall-clock windows is tested with an injected time on both sides of the window.
- S8 Freshness alarms on external feeds.** A five-hour network degradation on 30 September stalled Hyperliquid collection and Polymarket quotes at once; both recovered unnoticed. Each feed needs a staleness threshold that raises an event (D1) rather than a line in a log.
- S9 Multi-source intake** (H11). Sources beyond arXiv need per-source identifier namespaces, per-source watermarks, deduplication across sources by title and first author with aliases, and an explicit abstract-only status. SSRN qualifies only as metadata through Crossref (prefix 10.2139): its terms forbid automated access and full-text storage, so research prompts must forbid fetching from it. NBER and RePEc NEP feeds and further arXiv categories with a keyword filter are straightforward.
- S10 The operator view reads documents, not sources.** Learned from the statarb watchboard on 1 October. A unit's produced documents open as their PDF in a reader inside the page, with the $\text{\LaTeX}$ source one click away and a visible warning when the PDF is older than its source; a unit without a PDF falls back to its source text. Every unit's document has a stable link (`#note=<unit>`) so a PDF can be cited in a message. The page opens with the pipeline itself: one count per stage, split by state, each state a filter on the unit list. PDFs are served inline and may be framed only by the view's own origin.

8 Decisions taken

Settled by the operator on 29 September.

1. **Canon.** `pathfinder` is the canonical engine. `pathfinder-julien-2` is treated as a separate branch from which innovations are taken selectively (for example the read-only readers and hash-bound evidence index of its joint-reference candidate); it is not merged wholesale. This settles the frame for D8.
2. **Apex agent.** A strong Codex or Claude agent. It receives failure notifications, handles them, and resumes the campaign itself for most failure classes. Some classes are outside its remit: a usage limit is the reference case, where the right action is to pause until the reset and tell the operator, not to retry. The rule is to handle everything the agent can handle. It escalates only when the resolution lies outside its reach: a usage limit (wait for the reset), credentials it does not hold, or a decision that belongs to the operator, such as changing a research contract or scope. D6 and the `repres` skill follow this rule.
3. **Billing.** Subscription. Dollar cost is not a usable budget unit; budgets and accounting are in tokens and calls (D7), and usage limits are the binding constraint to detect and forecast.
4. **Operator view.** A web page, rendered from the campaign state of D1. The apex agent reads the same state directly, not the page. The page shows:
 - the campaign parameters (models, seats, rounds, allowances, EVA scheme and branch count, budget in tokens and calls);
 - progress so far: for each phase or stage (scan, research, edit, review and so on), the number of units of the batch, typically pairs, that are queued, running, done or blocked there;
 - blocks, grouped by failure class and cause;
 - a list of units with their scores, current stage and status, and links to the PDFs they produced, if any.
5. **Formats.** Logs and receipts need not stay compatible with earlier runs. D1 and D2 may define new formats freely, with no converter.

6. **Operator loop.** Every iteration contains the ten sections of the repres template as it stands, one file per iteration in `operator-log/`.
7. **Composable EVA in canon.** The julien-2 scheme (independent EVA-minus branches followed by a joint EVA thread over their bundles) is incorporated into `pathfinder` as an option, not as the default path. The number of branches is a campaign parameter with default 3. The measurements of D10 (thread divergence, Vera rejection rate) then inform the default rather than the existence of the option.
8. **Other lineages.** `statarb` and `agQSL` are first explored for variants worth reusing (for example `agQSL`'s quota detection, no-start timeout, size preflight and idle watcher, and `statarb`'s recovery diagnoser and implementation stage), then brought under the `pathfinder` roof as deployments of the canonical engine rather than separate copies.

All questions raised in this note are settled.